

Single Cycle RISC-V 32 Bits Microprocessor

Draft

Contents

Index	2
1 Introduction	4
2 RV32I Single-Cycle Microprocessor	4
2.1 Overview	4
2.2 Microprocessor - Port Conventions	5
2.3 RTL Snippet	5
2.4 RSICV-32 Top Diagram	11
3 Processor Specifications	12
3.1 Architectural Overview	12
3.2 Supported Instruction Types	12
3.3 Supported Arithmetic and Logical Operations	12
3.4 Supported Branch Operations	13
3.5 Supported Load and Store Operations	13
3.6 Control Flow and Program Counter Behavior	13
3.7 Behavior for Invalid or Unsupported Instructions	13
4 Verification Plan	14
4.1 Verification Objectives	14
4.2 Test Plan Strategy	14
4.3 Assertion-Based Verification Strategy	15
4.4 Verification Execution Model	17
5 Functional Coverage	17
5.1 Coverage Objectives	17
5.2 Cover Properties	17
5.3 Covergroups and Coverpoints	18
5.4 Coverage Report Evidence	19
6 Waveform Inspection in SimVision	20
7 Appendix A: RTL Modules - Most Significant	21
Appendix A: UART RTL Modules	21
7.1 control_unit - module	21

Draft

7.2	program_counter - module	25
7.3	transmitter - module	26
7.4	imm_gen - module	28
7.5	microprocessor_top_tb - module	29
7.6	fv_microprocessor_top - module	32

List of Figures

1	Architecture of Microprocessor.	11
2	Functional coverage summary corresponding to opcode-level and instruction-variant coverage.	19
3	Detailed covergroup bin coverage including <code>funct3</code> , <code>funct7</code> , and defined cross coverage.	20
4	SimVision waveform snapshot showing instruction flow and module-level behavior (PC, instruction decode, PRF write-back, ALU, ImmGen, branch decision, and data memory signals) for the RV32I single-cycle microprocessor.	21

Draft

Glossary

- **MSB (Most Significant Bit)** — The bit in a binary word with the highest positional weight. In a 32-bit RISC-V datapath, bit 31 is the MSB and determines the sign in signed operations.
- **LSB (Least Significant Bit)** — The bit in a binary word with the lowest positional weight. In a 32-bit architecture, bit 0 is the LSB and determines alignment and byte selection in memory operations.
- **RV32I** — The 32-bit base integer instruction set architecture (ISA) of RISC-V. It defines arithmetic, logical, branch, load/store, and control instructions implemented in this microprocessor.
- **Instruction Word** — A 32-bit encoded value fetched from instruction memory that contains opcode, register addresses, funct3, funct7, and immediate fields.
- **Opcode** — A field within the instruction word that determines the instruction type (R-type, I-type, S-type, B-type, U-type, J-type) and drives the control unit decoding logic.
- **Funct3 / Funct7** — Subfields of the instruction used to refine the operation type inside a given opcode group (e.g., ADD vs SUB differentiation).
- **Datapath** — The structural hardware path that processes data, including the Program Counter (PC), Register File, ALU, Immediate Generator, and memory interfaces.
- **Control Unit (CU)** — The combinational logic block responsible for decoding the instruction word and generating control signals such as ALU control, register write enable, memory write enable, and branch selection.
- **ALU (Arithmetic Logic Unit)** — The combinational block that performs arithmetic and logical operations (ADD, SUB, AND, OR, XOR, shifts, comparisons) based on control signals.
- **Register File (PRF)** — A bank of 32 general-purpose registers (x0–x31) used for operand storage and write-back operations. Register x0 is hardwired to zero.
- **Program Counter (PC)** — A register that holds the address of the current instruction. In a single-cycle architecture, PC is updated every clock cycle.
- **Immediate Generator (ImmGen)** — A combinational block that extracts and sign-extends immediate fields from the instruction word depending on the instruction format.
- **Branch Condition (BCOND)** — The logical comparison result used to determine whether a branch instruction modifies the PC.
- **Write-Back (WB)** — The stage in which the ALU or memory result is written into the destination register.
- **Single-Cycle Architecture** — A processor implementation in which each instruction completes in exactly one clock cycle.
- **Interface (SystemVerilog)** — A construct used in verification to group related signals and simplify DUT-testbench connectivity.
- **SVA (SystemVerilog Assertions)** — Formal properties used to verify protocol correctness, signal relationships, and architectural constraints (e.g., PC alignment, register write behavior).
- **Coverage** — Verification metric used to measure how thoroughly the instruction space and control paths have been exercised.

1 Introduction

This document presents the verification methodology and architectural overview of a custom-designed RV32I single-cycle RISC-V microprocessor. The primary objective of this project is to validate the functional correctness, control behavior, and datapath integrity of the processor at the RTL level using structured verification techniques.

The verification environment is built around a SystemVerilog-based testbench that applies controlled stimuli to the RTL design. Stimulus generation and signal observation are performed through a dedicated verification interface, enabling clean separation between the Design Under Test (DUT) and the testbench components.

The verification strategy includes:

- Directed and constrained instruction stimulus applied to the RTL processor.
- Signal abstraction and connectivity using SystemVerilog interfaces.
- Non-intrusive property checking using `bind` constructs.
- Formal and concurrent assertions (SystemVerilog Assertions, SVA) to validate architectural constraints and protocol correctness.
- Functional coverage using cover properties and covergroups to measure exercised instruction space and control paths.

The processor under verification implements the RV32I base integer instruction set architecture in a single-cycle configuration. Each instruction completes within one clock cycle, requiring precise coordination between the control unit, datapath elements, register file, ALU, immediate generator, and memory interfaces.

The following sections describe the architectural organization of the microprocessor, its RTL structure, verification approach, and assertion-based validation framework.

2 RV32I Single-Cycle Microprocessor

2.1 Overview

The `microprocessor_top` module represents the top-level integration of a custom-designed RV32I single-cycle RISC-V microprocessor. This module structurally connects all major architectural blocks required to execute one instruction per clock cycle.

The processor implements the base RV32I instruction set and integrates the following components:

- Program Counter (PC) and next-PC selection logic.
- Instruction Memory for instruction fetch.
- Register File (32 general-purpose registers).
- Immediate Generator.
- Arithmetic Logic Unit (ALU).

- Branch comparator and branch decision logic.
- Data Memory for load and store operations.
- Write-back selection multiplexer.
- Control Unit for instruction decoding and control signal generation.

The design follows a structural RTL approach, where this top-level module primarily consists of signal interconnections and submodule instantiations. All state elements are contained within dedicated submodules such as the program counter, register file, and memory blocks.

For verification purposes, the module also allows instruction injection through an external stimulus mechanism, enabling controlled execution scenarios during testbench-driven validation.

2.2 Microprocessor - Port Conventions

Port	Bits Used	Type	Description
clk	1	Input	Global synchronous clock driving all sequential elements of the processor.
arst_n	1	Input	Asynchronous active-low reset signal used to initialize state elements.
instr_en	1	Input	Instruction override enable. When asserted, the processor executes the externally driven instruction instead of the instruction fetched from memory.
driven_instr	[31:0]	Input	External instruction provided by the verification environment for controlled stimulus injection.

2.3 RTL Snippet

```

1 'timescale 1ns / 1ps
2 import riscv_params_pkg::*;
3 //
4 // -----
5 // Module: microprocessor_top
6 // Description:
7 //   Top-level integration for a single-cycle RV32-style microprocessor
8 //   datapath.
9 //   Contains:
10 //     - PC register and next-PC selection (sequential vs redirect)
11 //     - Instruction fetch (instruction memory)
12 //     - Register file read/write-back
13 //     - Immediate generation
14 //     - ALU execute
15 //     - Data memory (stores/loads when implemented)
16 //     - Control unit decode and branch decision
17 //
18 // Notes:
19 //   - Signal naming follows datapath intent: pc_* , if_* , id_* , ex_* , mem_* , wb_*
20 //   - This module is structural (wires + instantiations) and contains no state

```

```

19 //      besides submodules (PC, RF, memories).
20 //
-----
21 module microprocessor_top (
22     input logic          clk,
23     input logic          arst_n,
24     input logic          instr_en,
25     input logic [DATA_WIDTH-1:0] driven_instr
26 );
27
28
29 //
-----
30 // IF: Program Counter and Instruction Fetch
31 //
-----
32
33 logic [DATA_WIDTH-1:0] pc_q;          // current PC (byte address)
34 logic [DATA_WIDTH-1:0] pc_next;      // next PC selected by pc_next mux
35 logic [DATA_WIDTH-1:0] pc_plus_inc;  // sequential next PC (PC + inc)
36 logic                  pc_inc_sel;   // +4 vs +2 selector (future)
37 logic [2:0]            pc_inc;       // increment constant (2 or 4)
38
39 logic [DATA_WIDTH-1:0] instruction;
40 logic [DATA_WIDTH-1:0] mem_instruction; // fetched instruction
41                               Draft
42 // Next-PC select:
43 //   - branch_taken=0 -> sequential PC+4
44 //   - branch_taken=1 -> redirect target (computed in EX path)
45 logic                  branch_taken;
46 logic [DATA_WIDTH-1:0] branch_target; // target address (PC + imm),
47                               produced in EX
48
49 // PC next mux: select sequential or redirect target
50 mux #(.WIDTH(DATA_WIDTH)) upc_next_mux_i (
51     .in1(branch_target), // sel=1 -> redirect
52     .in2(pc_plus_inc),   // sel=0 -> sequential
53     .sel(branch_taken),
54     .out(pc_next)
55 );
56
57 // PC register
58 program_counter pc_i (
59     .clk      (clk),
60     .arst_n   (arst_n),
61     .pc_in    (pc_next),
62     .pc_out   (pc_q)
63 );
64
65 // PC increment select (+4 now; +2 reserved for future compressed support)
66 plus_4_or_2_mux pc_inc_sel_i (
67     .sel      (pc_inc_sel),
68     .instruction_add(pc_inc)

```

```

68     );
69
70     // PC + increment
71     adder #(.DATA_WIDTH(DATA_WIDTH)) pc_adder_i (
72         .constant_operand(pc_inc),
73         .instruction_addr(pc_q),
74         .next_instruction (pc_plus_inc)
75     );
76
77     // Instruction memory (word-indexed; PC is byte-addressed)
78     instruction_memory #(.DATA_WIDTH(DATA_WIDTH), .DEPTH(DEPTH)) instr_mem_i (
79         .pc      (pc_q),
80         .instr(mem_instruction)
81     );
82
83     //
84     -----
85
86     // ID: Decode and Register File Read
87     //
88     -----
89
90     logic                rf_we;
91     logic [DATA_WIDTH-1:0] rs1_data;
92     logic [DATA_WIDTH-1:0] rs2_data;
93
94     // Write-back data (selected later)
95     logic [DATA_WIDTH-1:0] wb_data; Draft
96
97     physical_register_file #(.DIR_WIDTH(DIR_WIDTH), .DATA_WIDTH(DATA_WIDTH))
98     prf_i (
99         .clk      (clk),
100        .arst_n    (arst_n),
101        .write_en   (rf_we),
102        .read_dir1  (instruction[19:15]), // rs1
103        .read_dir2  (instruction[24:20]), // rs2
104        .write_dir  (instruction[11:7]),  // rd
105        .write_data (wb_data),
106        .read_data1 (rs1_data),
107        .read_data2 (rs2_data)
108    );
109
110    //
111    -----
112
113    // ID/EX: Immediate Generation and Operand Select
114    //
115    -----
116
117    logic [2:0]                imm_sel;
118    logic [DATA_WIDTH-1:0] imm;
119
120    imm_gen imm_gen_i (
121        .instr      (instruction),

```



```

115     .imm_sel (imm_sel),
116     .imm_out (imm)
117 );
118
119 // ALU operand mux controls
120 logic [1:0]      op1_sel_pc;      // 00: use zero as operand 1, 01:
    use PC as operand 1, 10: use rs1 as operand 1
121 logic            op2_sel_imm;     // 1: use imm, 0: use rs2 as
    operand 2
122
123 logic [DATA_WIDTH-1:0] alu_op1;      // ALU operand 1
124 logic [DATA_WIDTH-1:0] alu_op2;      // ALU operand 2
125 logic [DATA_WIDTH-1:0] zero_operand = 32'd0; // Zero operand for LUI
    operation
126 // Operand1: PC vs rs1
127 mux_operand_1 alu_op1_mux_i (
128     .in1(zero_operand),    // Zero operand as operand 1
129     .in2(pc_q),            // PC as operand 1
130     .in3(rs1_data),        // rs1 as operand 1
131     .sel(op1_sel_pc),      // 00: use zero as operand 1, 01: use PC as operand
    1, 10: use rs1 as operand 1
132     .data_out(alu_op1)    // data_out as operand_1
133 );
134
135 // Operand2: imm vs rs2
136 mux #(.WIDTH(DATA_WIDTH)) alu_op2_mux_i (
137     .in1(imm),             // sel=1 -> imm
138     .in2(rs2_data),        // sel=0 -> rs2
139     .sel(op2_sel_imm),      Draft
140     .out(alu_op2)
141 );
142
143 //
    -----
144 // EX: ALU and Branch Decision
145 //
    -----
146
147 logic [3:0]      alu_ctrl;
148 logic [DATA_WIDTH-1:0] alu_result;
149
150 alu #(.OPERAND_WIDTH(DATA_WIDTH)) alu_i (
151     .operand1 (alu_op1),
152     .operand2 (alu_op2),
153     .alucontrol (alu_ctrl),
154     .alu_result (alu_result)
155 );
156
157 // For branches/jumps in this single-cycle design, branch_target is
    produced by ALU
158 // when operand1=PC and operand2=imm and alu_ctrl=ADD.
159 assign branch_target = alu_result;
160
161 // Comparator for TYPE-B conditions

```

```

162 logic b_condition_rs1_rs2;
163
164 branch #(.DATA_WIDTH_BRANCH(DATA_WIDTH)) branch_cmp_i (
165     .rs_1      (rs1_data),
166     .rs_2      (rs2_data),
167     .opcode     (instruction[6:0]),
168     .funct_3    (instruction[14:12]),
169     .branch_taken (b_condition_rs1_rs2)
170 );
171
172 //
173 -----
174
175 // MEM: Data Memory (store/load datapath)
176 //
177 -----
178
179 logic                                dmem_we;
180 logic [DATA_WIDTH-1:0] dmem_rdata;
181
182 data_memory #(.DATA_WIDTH(DATA_WIDTH), .DEPTH(DEPTH)) data_mem_i (
183     .clk      (clk),
184     .wr_en    (dmem_we),
185     .addr     (alu_result), // byte address from ALU
186     .data_in  (rs2_data),   // store data
187     .data_out (dmem_rdata)  // load data
188 );
189
190                                     Draft
191
192 //
193 -----
194
195 // WB: Write-back Mux (ALU vs MEM vs PC+4)
196 //
197 -----
198
199 logic [1:0] wb_sel;
200
201 mux_3_to_1 wb_mux_i (
202     .data_out_to_pc (pc_plus_inc), // PC+4 (link address)
203     .alu_to_mem_addr (alu_result), // ALU result
204     .data_out_to_mux (dmem_rdata), // memory read data
205     .sel             (wb_sel),
206     .data_out        (wb_data)
207 );
208
209 //
210 -----
211
212 // Control Unit: Decode instruction -> drive control signals
213 //
214 -----
215
216 control_unit cu_i (

```

```

206         .opcode           (instruction[6:0]),
207         .funct_3           (instruction[14:12]),
208         .funct_7           (instruction[31:25]),
209         .zero              (b_condition_rs1_rs2),      // B condition
210
211         .prf_wr_en         (rf_we),
212         .cu_imm_sel        (imm_sel),
213         .prf_pc_mux_ctrl   (op1_sel_pc),
214         .prf_imm_mux_ctrl  (op2_sel_imm),
215         .cu_alu_ctrl       (alu_ctrl),
216         .cu_mem_out_mux_sel(wb_sel),
217         .cu_data_mem_wr_en (dmem_we),
218         .cu_pc_add_sel     (pc_inc_sel),
219         .branch_taken      (branch_taken)
220     );
221
222     assign instruction = instr_en ? driven_instr : mem_instruction;
223
224 endmodule

```

Draft

2.4 RSICV-32 Top Diagram

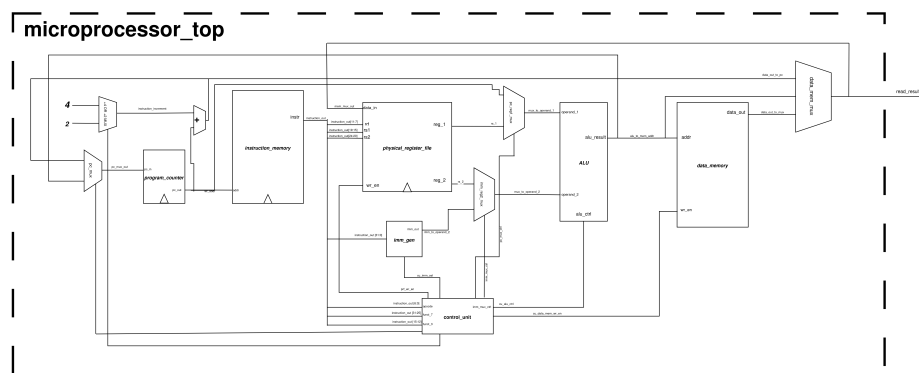


Figure 1: Architecture of Microprocessor.

Draft

3 Processor Specifications

3.1 Architectural Overview

The implemented processor is a 32-bit RV32I single-cycle RISC-V microprocessor. It executes one complete instruction per clock cycle, including instruction fetch, decode, execute, memory access (if required), and write-back.

The architecture is fully synchronous and operates under a single clock domain. All state elements (Program Counter, Register File, and memories) are updated on the rising edge of the clock.

The datapath width is 32 bits, and the processor supports the base integer instruction set defined by the RV32I specification.

3.2 Supported Instruction Types

The processor supports the following opcode groups:

- **R-Type** (0110011) – Register-to-register arithmetic and logical operations.
- **I-Type** (0010011) – Immediate arithmetic and logical operations.
- **L-Type** (0000011) – Load instructions.
- **S-Type** (0100011) – Store instructions.
- **B-Type** (1100011) – Conditional branch instructions.
- **U-Type LUI** (0110111) – Load Upper Immediate.
- **U-Type AUIPC** (0010111) – Add Upper Immediate to PC.
- **J-Type (JAL)** (1101111) – Jump and Link.

3.3 Supported Arithmetic and Logical Operations

For R-type and I-type instructions, the following `funct3`-based operations are supported:

- ADD / SUB
- SLL (Shift Left Logical)
- SLT (Set Less Than, signed)
- SLTU (Set Less Than, unsigned)
- XOR
- SRL / SRA (Shift Right Logical / Arithmetic)
- OR
- AND

3.4 Supported Branch Operations

The processor supports the following conditional branch instructions:

- BEQ (Branch if Equal)
- BNE (Branch if Not Equal)
- BLT (Branch if Less Than, signed)
- BGE (Branch if Greater or Equal, signed)
- BLTU (Branch if Less Than, unsigned)
- BGEU (Branch if Greater or Equal, unsigned)

3.5 Supported Load and Store Operations

Load instructions:

- LB (Load Byte, signed)
- LH (Load Halfword, signed)
- LW (Load Word)
- LBU (Load Byte, unsigned) Draft
- LHU (Load Halfword, unsigned)

Store instructions:

- SB (Store Byte)
- SH (Store Halfword)
- SW (Store Word)

3.6 Control Flow and Program Counter Behavior

The Program Counter (PC) is byte-addressed and normally increments by 4 for sequential instruction execution. When a branch or jump condition is satisfied, the PC is redirected to the computed target address.

3.7 Behavior for Invalid or Unsupported Instructions

If the control unit receives an instruction with an unsupported or undefined opcode, the processor shall not stall or enter an undefined state.

Instead, the following behavior is expected:

- The instruction is treated as a NOP (No Operation).

- The PC continues to increment sequentially ($PC + 4$).
- No register write or memory write operation is performed.

In this implementation, the default NOP behavior corresponds to an `ADDI x0, x0, 0` instruction, which produces no architectural state change and ensures forward program progress.

This requirement guarantees forward progress and prevents deadlock conditions caused by invalid instruction decoding.

4 Verification Plan

4.1 Verification Objectives

The objective of this verification plan is to validate the functional correctness, control integrity, and architectural compliance of the RV32I single-cycle microprocessor.

Verification focuses on ensuring:

- Correct execution of all supported legal instructions.
- Proper Program Counter (PC) behavior under sequential and control-flow instructions.
- Correct register file behavior, including preservation of `x0 = 0`.
- Correct ALU computation for arithmetic and logical operations.
- Correct memory interface behavior for load and store instructions.
- Safe forward progress in the presence of illegal or unsupported instructions.

The verification strategy is assertion-driven and instruction-aware, meaning that properties are evaluated dynamically according to the decoded instruction type.

4.2 Test Plan Strategy

The processor is verified using the following stimulus categories:

1. Directed Legal Instruction Testing

The processor is stimulated with all supported legal instruction encodings. Each opcode and its associated `funct3` and `funct7` combinations are exercised.

The goal of this phase is to:

- Validate datapath correctness.
- Validate control signal generation.
- Verify write-back correctness.
- Confirm correct branch and jump redirection.

2. Illegal Instruction Testing

The processor is stimulated with unsupported or invalid opcode encodings.

Expected behavior:

- The processor must not stall or enter an undefined state.
- No unintended register or memory write shall occur.
- The PC shall continue sequential execution ($PC + 4$).

This test ensures architectural robustness and forward progress.

3. Constrained Random Legal Instruction Testing

Randomly generated legal instructions are applied to the processor.

The objective is to:

- Stress control logic transitions.
- Exercise diverse register and immediate combinations.
- Detect corner-case arithmetic or comparison errors.
- Validate cumulative execution behavior over extended instruction streams.

Draft

Random testing increases confidence beyond directed stimulus by exploring unexpected operand combinations.

4.3 Assertion-Based Verification Strategy

Verification is performed using SystemVerilog Assertions (SVA). Assertions are bound non-intrusively to the DUT and are evaluated for every decoded instruction.

Each assertion is conditionally activated based on the current instruction type. This ensures that properties are instruction-aware and context-sensitive.

The assertions are grouped as follows:

1. Global Architectural Properties

These properties must hold for all instructions:

- **PC alignment:** The Program Counter must always be word-aligned.
- **Sequential progression:** Non-branch instructions must advance PC by 4.
- **Register x0 invariant:** Register x0 must always remain zero.

2. I-Type Instruction Properties

For I-type arithmetic instructions:

- Immediate selection must correspond to I-format.
- Register write-enable must be asserted.
- Write-back data must equal the expected ALU computation.

Assertions validate:

- ADDI result correctness.
- SLTIU comparison behavior.
- XORI, ORI, ANDI bitwise correctness.

3. R-Type Instruction Properties

For register-to-register instructions:

- Correct arithmetic behavior (ADD, SUB).
- Correct shift operations (SLL, SRL, SRA).
- Correct signed and unsigned comparisons (SLT, SLTU).
- Correct bitwise logic (AND, OR, XOR). Draft

Each operation has a dedicated property that verifies write-back data against the expected ALU result.

4. Control-Flow Instruction Properties

For control instructions:

- JAL must redirect PC to the computed target.
- AUIPC must write $PC + \text{immediate}$.
- LUI must correctly place the immediate in the upper bits.
- Branch instructions must update PC conditionally based on comparison results.

Branch assertions explicitly verify both taken and not-taken paths.

5. Memory Instruction Properties

For memory operations:

- Store instructions must assert data memory write enable.
- Store address must equal $RS1 + IMM$.
- Load instructions must disable memory write.
- Load operations must write memory data back into the register file.

4.4 Verification Execution Model

Assertions are evaluated on every decoded instruction using an instruction-triggered property macro structure. This ensures:

- Deterministic checking per instruction.
- Immediate failure detection.
- Clear localization of functional errors.

This assertion-centric strategy provides cycle-accurate validation of datapath, control logic, and architectural invariants.

5 Functional Coverage

5.1 Coverage Objectives

In addition to assertion-based checking, functional coverage is collected to quantify how thoroughly the verification stimulus exercises the supported instruction space and key decode/execute combinations.

Coverage is used to confirm that:

- All supported opcode groups (R/I/L/S/B/LUI/AUIPC/JAL) are observed during simulation.
- The relevant `funct3` variants for I-type and R-type instructions are exercised.
- R-type `funct7` variants are exercised, specifically ensuring SUB and SRA decode paths are hit.
- Cross-coverage between `funct3` and `funct7` is achieved for R-type instructions.
- Illegal/unsupported instruction activity can be detected (either directly or through the NOP mapping behavior).

The resulting coverage reports are collected from simulation and documented at the end of this section using screenshots of the generated coverage summary.

5.2 Cover Properties

Cover properties are defined to confirm that each major instruction class is observed at least once. These covers are instruction-driven and trigger when the opcode matches the corresponding group.

```
1 // -----
2 // COVERS
3 // -----
4 'COV(it, cov_any_i_type, ('OPCODE == OPCODE_I_TYPE) |->, 1'b1)
5 'COV(it, cov_any_r_type, ('OPCODE == OPCODE_R_TYPE) |->, 1'b1)
6 'COV(it, cov_any_b_type, ('OPCODE == OPCODE_B_TYPE) |->, 1'b1)
7 'COV(it, cov_any_l_type, ('OPCODE == OPCODE_L_TYPE) |->, 1'b1)
8 'COV(it, cov_any_s_type, ('OPCODE == OPCODE_S_TYPE) |->, 1'b1)
9 'COV(it, cov_any_lui, ('OPCODE == OPCODE_U_LUI) |->, 1'b1)
10 'COV(it, cov_any_auipc, ('OPCODE == OPCODE_U_AUIPC) |->, 1'b1)
11 'COV(it, cov_any_jal, ('OPCODE == OPCODE_JAL_TYPE) |->, 1'b1)
```

5.3 Covergroups and Coverpoints

To obtain finer-grain insight beyond opcode-level observation, a covergroup is defined and sampled on every clock cycle after reset. This covergroup captures opcode distribution, instruction variants through `funct3` and `funct7`, and key cross-coverage for R-type instructions.

```
1 // -----
2 // COVERGROUPS
3 // -----
4
5 // Sample coverage on every cycle after reset
6 covergroup cg_instr @(posedge clk);
7     option.per_instance = 1;
8
9     // Cover that we ever saw a NOP on the instruction bus
10    cp_nop: coverpoint ('OPCODE == 7'b1111111) iff (arst_n) {
11        bins seen_nop = {1'b1};
12    }
13
14    // Basic opcode coverage
15    cp_opcode: coverpoint 'OPCODE iff (arst_n) {
16        bins r_type = {OPCODE_R_TYPE};
17        bins i_type = {OPCODE_I_TYPE};
18        bins l_type = {OPCODE_L_TYPE};
19        bins s_type = {OPCODE_S_TYPE};
20        bins b_type = {OPCODE_B_TYPE};
21        bins lui    = {OPCODE_U_LUI};
22        bins auipc  = {OPCODE_U_AUIPC};
23        bins jal    = {OPCODE_JAL_TYPE}; Draft
24        // If illegal opcodes are mapped to NOP by the control logic, track them
25        // as "other"
26        bins other  = default;
27    }
28
29    // I-type funct3 coverage (only meaningful for I-type)
30    cp_funct3_i: coverpoint 'FUNCT3 iff (arst_n && ('OPCODE == OPCODE_I_TYPE)) {
31        bins addi  = {F3_ADD_SUB};
32        bins slti  = {F3_SLT};
33        bins sltiu = {F3_SLTU};
34        bins xori  = {F3_XOR};
35        bins ori   = {F3_OR};
36        bins andi  = {F3_AND};
37    }
38
39    // R-type funct3 coverage (only meaningful for R-type)
40    cp_funct3_r: coverpoint 'FUNCT3 iff (arst_n && ('OPCODE == OPCODE_R_TYPE)) {
41        bins add_sub = {F3_ADD_SUB};
42        bins sll     = {F3_SLL};
43        bins slt     = {F3_SLT};
44        bins sltu    = {F3_SLTU};
45        bins xor_    = {F3_XOR};
46        bins srl_sra = {F3_SRL_SRA};
47        bins or_     = {F3_OR};
48        bins and_    = {F3_AND};
49    }
```

```

50 // R-type funct7 coverage (SUB/SRA differentiation)
51 cp_funct7_r: coverpoint 'FUNCT7 iff (arst_n && ('OPCODE == OPCODE_R_TYPE)) {
52     bins base = {7'b0000_000};
53     bins sub  = {7'b0100_000};
54 }
55
56 // Cross-coverage: funct3 x funct7 for R-type
57 cx_rtype: cross cp_funct3_r, cp_funct7_r iff (arst_n && ('OPCODE ==
58     OPCODE_R_TYPE));
59 endgroup
60
61 cg_instr cg_instr_i;
62
63 initial begin
64     // Create coveragegroup instance
65     cg_instr_i = new();
66 end

```

5.4 Coverage Report Evidence

The final coverage results are obtained from the simulator coverage report. Screenshots of the coverage summary and detailed bins type are included below.

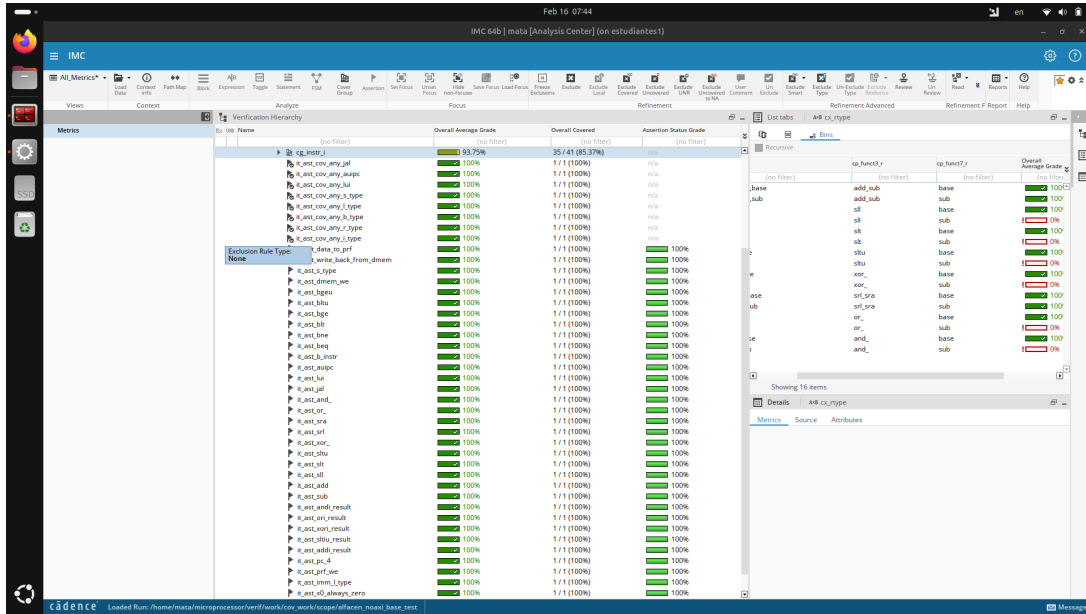


Figure 2: Functional coverage summary corresponding to opcode-level and instruction-variant coverage.

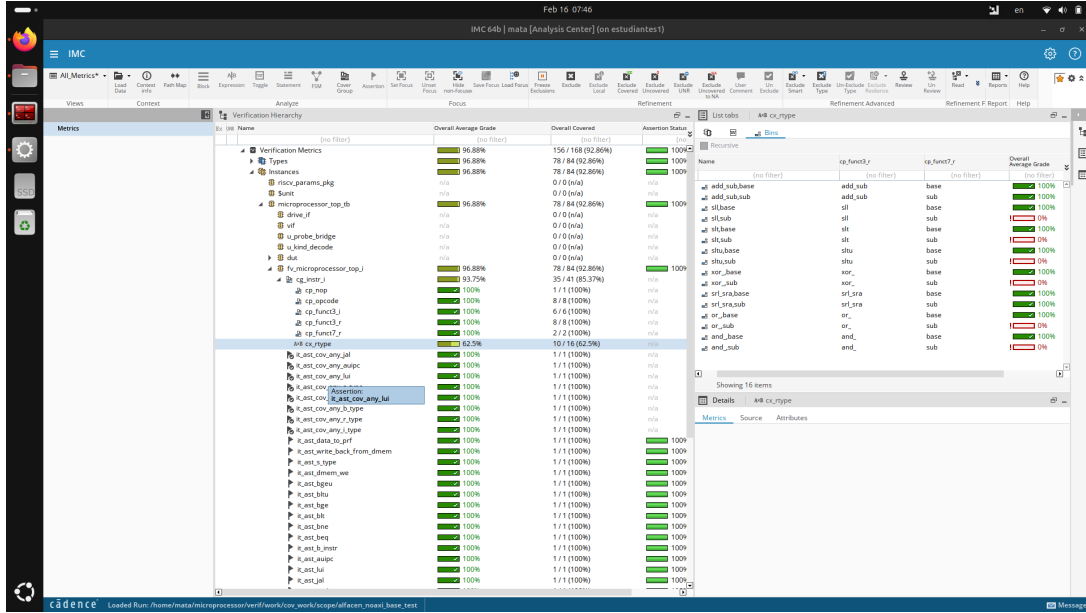


Figure 3: Detailed covergroup bin coverage including funct3, funct7, and defined cross coverage.

6 Waveform Inspection in SimVision

Waveform inspection was performed in Cadence ^{Draft} SimVision to visually validate correct instruction sequencing and module-level behavior across the single-cycle datapath. Figure 4 shows the clock and reset, the decoded instruction stream, and the main datapath signals (PC update, register file activity, ALU operands/results, immediate generation, branch decision, and data memory control). The captured waveforms provide qualitative confirmation that each decoded instruction drives the expected control and datapath transitions within one clock cycle.

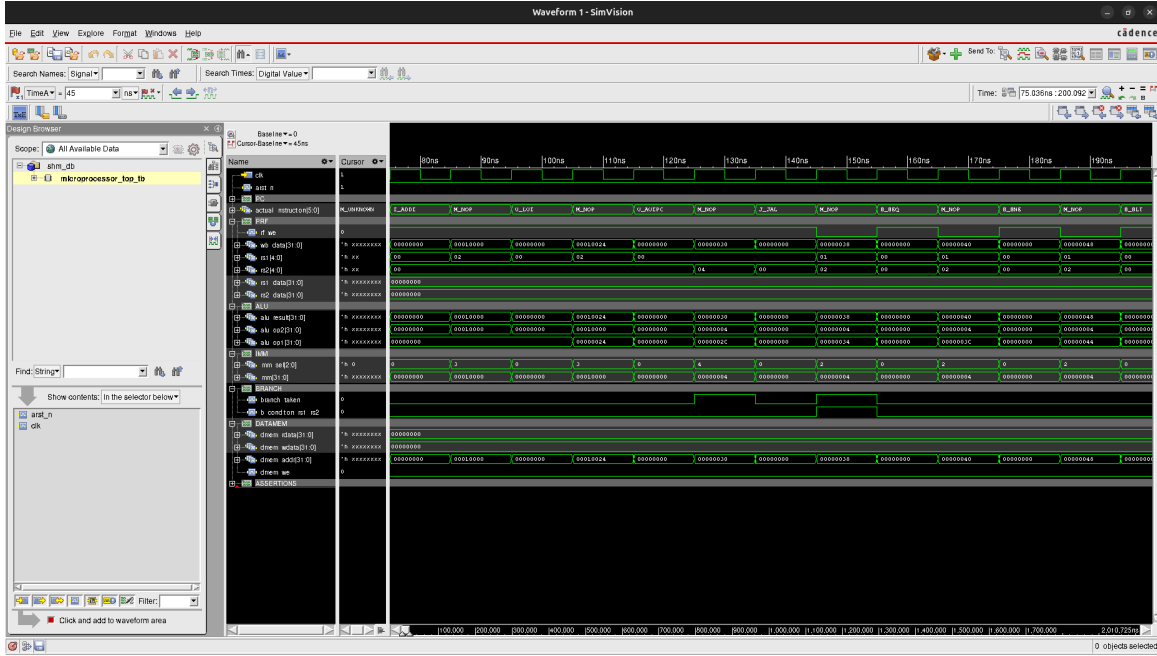


Figure 4: SimVision waveform snapshot showing instruction flow and module-level behavior (PC, instruction decode, PRF write-back, ALU, ImmGen, branch decision, and data memory signals) for the RV32I single-cycle microprocessor.

Draft

7 Appendix A: RTL Modules - Most Significant

7.1 control_unit - module

```

1 'timescale 1ns / 1ps
2 //
3
4 // Module: control_unit
5 // Description:
6 //   Combinational control unit for a single-cycle RV32I-style datapath.
7 //   Decodes {opcode, funct3, funct7} and generates control signals for:
8 //     - Register file write-back enable and write-back source selection
9 //     - Immediate format selection (imm_gen control)
10 //     - ALU operation selection
11 //     - Data memory write enable (stores)
12 //     - PC update decision (sequential vs branch/jump target)
13 //
14 // Supported instructions (current):
15 //   - I-type: ADDI, SLTI, SLTIU, XORI, ORI, ANDI
16 //   - R-type: ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
17 //   - B-type: BEQ (branch decision uses input 'zero')
18 //   - J-type: JAL (unconditional)
19 //
20 // Assumptions:
21 //   - 'zero' is asserted when rs1 == rs2 (BEQ compare evaluated externally).
```

```

21 // - Control encodings (IMM_*, ALU_*, mux selects, opcodes) are defined in
    defines.svh.
22 // - PC-target computation (PC + imm) and PC muxing are implemented outside
    this module.
23 //
24 // Notes:
25 // - Default assignments implement the common I-type ALU-immediate path.
26 // - Opcode decoding below overrides only what differs per instruction class
    .
27 // - Assertions for illegal/unsupported encodings can be added in a later
    stage.
28 //
    -----

29 import riscv_params_pkg::*;
30 module control_unit(
31     input logic [6:0] opcode,           // instr[6:0]
32     input logic [2:0] funct_3,          // instr[14:12]
33     input logic [6:0] funct_7,          // instr[31:25]
34     input logic zero,                   // 1 when rs1 == rs2 (BEQ
        condition)
35     output logic prf_wr_en,              // register file write enable
36     output logic [2:0] cu_imm_sel,       // immediate format select (
        imm_gen)
37     output logic [1:0] prf_pc_mux_ctrl,  // ALU operand1 select (rs1 vs PC
        )
38     output logic prf_imm_mux_ctrl,       // ALU operand2 select (rs2 vs
        imm)
39     output logic [3:0] cu_alu_ctrl, Draft // ALU operation select
40     output logic [1:0] cu_mem_out_mux_sel, // write-back mux select (to PRF)
41     output logic cu_data_mem_wr_en,      // data memory write enable (
        stores)
42     output logic cu_pc_add_sel,           // PC increment select (+4 now;
        +2 reserved)
43     output logic branch_taken            // PC redirect request (branch/
        jump)
44 );
45
46 always_comb begin
47     //
        -----

48     // Defaults: I-type ALU-immediate behavior
49     // - rd write enabled
50     // - ALU computes rs1 + imm
51     // - PC advances sequentially (+4)
52     // Opcode decode below overrides these defaults as required.
53     //
        -----

54     prf_wr_en          = 1'b1;
55     cu_data_mem_wr_en  = 1'b0;
56     cu_imm_sel         = IMM_I;
57     prf_pc_mux_ctrl    = 2'b10;           // operand1 = rs1
58     prf_imm_mux_ctrl   = 1'b1;           // operand2 = imm

```

```

59     cu_pc_add_sel      = 1'b0;          // +4 in RV32; +2 reserved for
        future compressed support
60     cu_mem_out_mux_sel = ALU_TO_PRF;    // write-back = ALU result
61     cu_alu_ctrl        = ALU_ADD;
62     branch_taken       = 1'b0;          // default: no PC redirect
63
64     unique case (opcode)
65
66         // I-type ALU-immediate ops: rd <- rs1 op imm
67         OPCODE_I_TYPE: begin
68             unique case (funct_3)
69                 F3_ADD_SUB: cu_alu_ctrl = ALU_ADD;
70                 F3_SLL:    cu_alu_ctrl = ALU_SLT;
71                 F3_SLTU:   cu_alu_ctrl = ALU_SLTU;
72                 F3_XOR:    cu_alu_ctrl = ALU_XOR;
73                 F3_OR:     cu_alu_ctrl = ALU_OR;
74                 F3_AND:    cu_alu_ctrl = ALU_AND;
75                 default: cu_alu_ctrl = ALU_ADD;
76             endcase
77         end
78
79         // B-type branch: no write-back; PC redirected when branch_taken=1
80         OPCODE_B_TYPE: begin
81             prf_wr_en      = 1'b0;      // branches do not write rd
82             cu_imm_sel     = IMM_B;      // branch offset immediate
83             prf_pc_mux_ctrl = 2'b01;    // operand1 = rs1 (as defined by
                your datapath for target calc)
84             cu_alu_ctrl    = ALU_ADD;    // used for target computation
                elsewhere (PC + imm path)
85             branch_taken   = zero;      // redirect request when condition is
                true
86             unique case (funct_3)
87                 // BEQ: take branch when rs1 == rs2 (zero=1)
88                 F3_BEQ: ;
89                 // BNE: take branch when rs1 != rs2 (zero=1)
90                 F3_BNE: ;
91                 // BLT: take branch when rs1 < rs2 (zero=1)
92                 F3_BLT: ;
93                 // BGE: take branch when rs1 >= rs2 (zero=1)
94                 F3_BGE: ;
95                 // BLTU: take branch when rs1 < rs2 (zero=1)
96                 F3_BLTU: ;
97                 // BGEU: take branch when rs1 >= rs2 (zero=1)
98                 F3_BGEU: ;
99                 default: begin
100                     cu_alu_ctrl = ALU_ADD;
101                     branch_taken = 1'b0;
102                 end
103             endcase
104         end
105
106         // R-type ALU-register ops: rd <- rs1 op rs2
107         OPCODE_R_TYPE: begin
108             prf_imm_mux_ctrl = 1'b0;    // operand2 = rs2 (not immediate)
109
110             // funct7 selects ADD/SUB and SRL/SRA families in RV32I

```



```

111         if (funct_7 == 7'b0000_000) begin
112             unique case (funct_3)
113                 F3_ADD_SUB:    cu_alu_ctrl = ALU_ADD;
114                 F3_SLL:      cu_alu_ctrl = ALU_SLL;
115                 F3_SLT:      cu_alu_ctrl = ALU_SLT;
116                 F3_SLTU:     cu_alu_ctrl = ALU_SLTU;
117                 F3_XOR:      cu_alu_ctrl = ALU_XOR;
118                 F3_SRL_SRA:   cu_alu_ctrl = ALU_SRL;
119                 F3_OR:       cu_alu_ctrl = ALU_OR;
120                 F3_AND:      cu_alu_ctrl = ALU_AND;
121                 default: cu_alu_ctrl = ALU_ADD;
122             endcase
123         end else if (funct_7 == 7'b0100_000) begin
124             unique case (funct_3)
125                 F3_ADD_SUB:    cu_alu_ctrl = ALU_SUB;
126                 F3_SRL_SRA:   cu_alu_ctrl = ALU_SRA;
127                 default: cu_alu_ctrl = ALU_ADD;
128             endcase
129         end else begin
130             cu_alu_ctrl = ALU_ADD; // unsupported funct7 -> safe
131                                     default
132         end
133
134         // JAL: rd <- PC+4, PC <- PC + imm (unconditional redirect)
135         OPCODE_JAL_TYPE: begin
136             cu_imm_sel          = IMM_J;                // jump offset
137             immediate          = 0;                    // operand1 = PC (
138                                     for target calc path)
139             cu_mem_out_mux_sel = INSTRUCTION_TO_PRF; // write-back
140                                     selects PC+4 (per your datapath naming)
141             cu_alu_ctrl        = ALU_ADD;
142             branch_taken       = 1'b1;                // unconditional
143                                     redirect
144         end
145
146         // load-word
147         OPCODE_L_TYPE: begin
148             cu_mem_out_mux_sel = DATA_OUT_TO_PRF;
149             unique case (funct_3)
150                 F3_LB:        ;
151                 F3_LH:        ;
152                 F3_LW:        ;
153                 F3_LBU:       ;
154                 F3_LHU:       ;
155                 default: prf_wr_en = 1'b1;
156             endcase
157         end
158
159         // write-word
160         OPCODE_S_TYPE: begin
161             prf_wr_en = 1'b0;
162             cu_mem_out_mux_sel = ALU_TO_PRF;
163             cu_imm_sel        = IMM_S;
164             cu_data_mem_wr_en = 1'b1;
165             unique case (funct_3)
166                 F3_SB:        ;

```

```

162         F3_SH:      ;
163         F3_SW:      ;
164         default: prf_wr_en = 1'b0;
165     endcase
166 end
167 // LUI operation
168 OPCODE_U_LUI: begin
169     prf_pc_mux_ctrl    = 2'b00;           // operand1 = PC
170     cu_imm_sel         = IMM_U;           // Extend the width
171     of imm
172 end
173 // AUIPC operation
174 OPCODE_U_AUIPC: begin
175     prf_pc_mux_ctrl    = 2'b01;           // operand1 = PC
176     cu_imm_sel         = IMM_U;           // Extend the width
177     of imm
178 end
179 // Default: NOP
180 default: begin
181     prf_wr_en          = 1'b0;
182     cu_alu_ctrl        = ALU_ADD;
183     branch_taken       = 1'b0;
184 end
185 endcase
186 end
187 endmodule

```

Draft

7.2 program_counter - module

```

1 'timescale 1ns / 1ps
2 //
3 // -----
4 // Module: program_counter
5 // Description:
6 //   Program counter (PC) register. Captures the next PC value on each rising
7 //   clock edge and provides the current PC to the instruction fetch path.
8 //
9 // Assumptions:
10 //   - arst_n is an active-low asynchronous reset.
11 //
12 // Notes:
13 //   - Reset initializes PC to 0x0000_0000.
14 //   - This block is purely sequential storage; PC update selection is handled
15 //     elsewhere in the datapath/control.
16 // -----
17
18 module program_counter #(
19     parameter DATA_WIDTH = 32,
20     parameter [DATA_WIDTH-1:0] RESET_PC = '0
21 )(
22     input logic clk,
23     input logic arst_n,

```

```

22     input logic [DATA_WIDTH - 1:0] pc_in,
23     output logic [DATA_WIDTH - 1:0] pc_out
24 );
25 //Secuential logic
26 always_ff @(posedge clk, negedge arst_n) begin
27     if (!arst_n) begin
28         pc_out <= RESET_PC; //Reset PC to boot address
29     end else begin
30         pc_out <= pc_in;
31     end
32 end
33 endmodule

```

7.3 transmitter - module

```

1
2 'timescale 1ns / 1ps
3 //
4 // Company:
5 // Engineer: Manuel Enrique Rivera Acosta
6 //
7 // Create Date: 10/28/2025 12:38:11 PM
8 // Design Name:
9 // Module Name: transmitter
10 // Project Name: Draft
11 //
12
13
14 module transmitter #(parameter BYTE_WIDTH = 8)(
15     input logic clk, // clock signal
16     input logic arst_n, // asynchronous reset
17     input logic [BYTE_WIDTH - 1 : 0] data_in, // DATA_IN TO BE SENT TO THE
18         RECEIVER MODULE BIT BY BIT THROUGH THE TX
19     input logic tick, // TICK COMING FROM THE BAUD RATE GENERATOR
20     input logic tx_start, // TX_START SIGNAL TO BEGIN THE BYTE TRANSMISSION
21     output logic tx, // SERIAL OUTPUT BIT TO TRANSMIT INTO THE RECEIVER RX
22         INPUT SIGNAL
23     output logic tx_done // DONE SIGNAL INDICATING THAT THE BYTE HAS BEEN SENT
24 );
25
26 localparam BIT_SAMPLING = 15; // localparam for the oversampling counter of
27     the uart x16
28
29 logic [BYTE_WIDTH - 1 : 0] shifted_data; // temporal register to store the
30     data shifted to the tx
31 logic [BYTE_WIDTH - 1 : 0] shifted_data_next; // next-state temporal register
32     shifted_data
33
34 logic [4:0] nbits; // variable to count the number of bits that has been sent
35 logic [4:0] nbits_next; // next-state nbits variable
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

32 logic [4:0] oversampling_count; // oversampling counter for the uart x16
33 logic [4:0] oversampling_count_next; // next-state oversampling counter
34
35 logic tx_done_next; // next-state tx_done signal (transmission done signal)
36 logic tx_next; // next-state tx signal (serial output bit)
37
38 typedef enum logic [2:0] {IDLE = 3'b000, START = 3'b001, DATA = 3'b010, STOP
    = 3'b011} state_type;
39
40 state_type state_reg, state_next;
41
42 always_ff @(posedge clk or negedge rst_n) begin
43     if(!rst_n) begin
44         state_reg <= IDLE;
45     end else begin
46         state_reg <= state_next;
47         nbits <= nbits_next;
48         oversampling_count <= oversampling_count_next;
49         shifted_data <= shifted_data_next;
50         tx_done <= tx_done_next;
51         tx <= tx_next;
52     end
53 end
54
55 always_comb begin
56     state_next = state_reg;
57     nbits_next = nbits;
58     oversampling_count_next = oversampling_count;
59     shifted_data_next = shifted_data;
60     tx_done_next = tx_done;
61     tx_next = tx;
62
63     case(state_reg)
64
65         IDLE: begin
66             tx_done_next = 1'b0;
67             nbits_next = '0;
68             tx_next = 1'b1;
69             oversampling_count_next = '0;
70             if(tx_start) begin
71                 shifted_data_next = data_in;
72                 tx_next = 1'b0;
73                 state_next = START;
74             end
75         end
76
77         START: begin
78             if(tick) begin
79                 if(oversampling_count == BIT_SAMPLING) begin
80                     oversampling_count_next = '0;
81                     state_next = DATA;
82                 end else begin
83                     oversampling_count_next = oversampling_count + 1;
84                 end
85             end
86         end

```

```

87
88     DATA: begin
89         tx_next = shifted_data[0];
90         if(tick) begin
91             if(oversampling_count == BIT_SAMPLING) begin
92                 shifted_data_next = {1'b0, shifted_data[7:1]};
93                 oversampling_count_next = '0;
94                 if(nbits == BYTE_WIDTH - 1) begin
95                     state_next = STOP;
96                 end else begin
97                     nbits_next = nbits + 1'b1;
98                 end
99             end else begin
100                 oversampling_count_next = oversampling_count + 1;
101             end
102         end
103     end
104
105     STOP: begin
106         tx_next = 1'b1;
107         if(tick) begin
108             if(oversampling_count == BIT_SAMPLING) begin
109                 state_next = IDLE;
110                 tx_done_next = 1'b1;
111             end else begin
112                 oversampling_count_next = oversampling_count + 1;
113                 state_next = STOP;
114             end
115         end
116     end
117     default: begin
118         state_next = state_reg;
119     end
120 endcase
121 end
122
123 endmodule

```

7.4 imm_gen - module

```

1  'timescale 1ns / 1ps
2  import riscv_params_pkg::*;
3  //
4  // -----
5  // Module: imm_gen
6  // Description:
7  //   RISC-V RV32I immediate generator. Extracts and sign/zero-extends the
8  //   immediate field from a 32-bit instruction according to imm_sel.
9  // Assumptions:
10 //   - instr is a valid RV32I instruction word.
11 //   - imm_sel is provided by the control unit and selects the immediate
12 //   format:
13 //       IMM_I: I-type    (sign-extended 12-bit immediate)

```

```

13 //      IMM_S: S-type      (sign-extended 12-bit store immediate)
14 //      IMM_B: B-type      (sign-extended branch offset, LSB forced to 0)
15 //      IMM_U: U-type      (upper immediate, lower 12 bits are 0)
16 //      IMM_J: J-type      (sign-extended jump offset, LSB forced to 0)
17 //
18 // Notes:
19 //   - Branch/jump immediates are instruction-aligned; therefore bit[0] is 0.
20 //   - This block performs field extraction/extension only. Target address
21 //     computation (PC + imm) is performed elsewhere.
22 //
-----
23 module imm_gen(
24     input  logic [31:0] instr,
25     input  logic [2:0]  imm_sel,
26     output logic [31:0] imm_out
27 );
28
29     always_comb begin
30         imm_out = '0; // default
31
32         unique case (imm_sel)
33
34             // I-type immediate: imm[11:0] = instr[31:20], sign-extended to 32
35             //                                     b
36             IMM_I: imm_out = {{20{instr[31]}}, instr[31:20]};
37
38             // S-type immediate: imm[11:5]=instr[31:25], imm[4:0]=instr[11:7],
39             //                                     sign-extended      Draft
40             IMM_S: imm_out = {{20{instr[31]}}, instr[31:25], instr[11:7]};
41
42             // B-type immediate (branch offset):
43             // imm[12]=instr[31], imm[11]=instr[7], imm[10:5]=instr[30:25],
44             // imm[4:1]=instr[11:8], imm[0]=0
45             IMM_B: imm_out = {{19{instr[31]}}, instr[31], instr[7], instr
46                 [30:25], instr[11:8], 1'b0};
47
48             // U-type immediate: imm[31:12]=instr[31:12], lower 12 bits are
49             //                                     zero
50             IMM_U: imm_out = {instr[31:12], 12'b0};
51
52             // J-type immediate (jump offset):
53             // imm[20]=instr[31], imm[19:12]=instr[19:12], imm[11]=instr[20],
54             // imm[10:1]=instr[30:21], imm[0]=0
55             IMM_J: imm_out = {{11{instr[31]}}, instr[31], instr[19:12], instr
56                 [20], instr[30:21], 1'b0};
57
58             default: imm_out = '0;
59         endcase
60     end
61 endmodule

```

7.5 microprocessor_top_tb - module

```

1 'timescale 1ns/1ps
2
3 //import riscv_params_pkg::*;
4
5 module microprocessor_top_tb;
6
7 //
8 // -----
9 // Clock / Reset
10 // -----
11
12 bit clk;
13 bit arst_n;
14
15 initial clk = 1'b0;
16 always #5ns clk = ~clk;
17
18 initial begin
19     arst_n = 1'b0;
20     repeat (3) @(posedge clk);
21     arst_n = 1'b1;
22 end
23
24 //
25 // -----
26 // Draft
27 // -----
28 // Interfaces
29 // -----
30
31 instr_drive_if #(DATA_WIDTH) drive_if (clk);
32 microprocessor_if #(DATA_WIDTH, DIR_WIDTH) vif (clk);
33 microprocessor_probe_bridge u_probe_bridge (.vif(vif));
34
35 riscv32_rand_instruction tr;
36
37 instr_kind_decode_for_waves u_kind_decode (.vif(vif));
38
39 //
40 // -----
41 // DUT
42 // -----
43
44 microprocessor_top dut (
45     .clk          (clk),
46     .arst_n       (arst_n),
47     .instr_en     (drive_if.instr_en),
48     .driven_instr (drive_if.driven_instr)
49 );
50
51 //
52 // -----

```

```

44 // CHECKER
45 //
-----

46 fv_microprocessor_top #(
47     .DATA_W(DATA_WIDTH),
48     .DIR_W(DIR_WIDTH)
49 ) fv_microprocessor_top_i (
50     .clk      (clk),
51     .arst_n(arst_n),
52     .vif      (vif)
53 );
54
55 //
=====

56 // Initialize
57 //
=====

58 initial begin : init_state
59     for (int i = 1; i < 32; i++) begin
60         dut.prf_i.prf[i] = '0;
61     end
62
63     for (int j = 0; j < DEPTH; j++) begin
64         dut.data_mem_i.mem[j] = '0;
65     end
66 end
67
68 //
=====

69 // TEST_PROGRAM
70 //
=====

71 initial begin : test_program
72     tr = new();
73
74     // -----
75     // Wait until reset is released
76     // -----
77     wait (arst_n == 1'b1);
78     repeat (2) @(posedge clk);
79
80     // -----
81     // 1) Directed sweep: hit every legal instruction at least once
82     // -----
83     drive_if.drive_all_legal_once();
84
85     // Optional idle cycle
86     @(posedge clk);
87
88     // -----

```



```

89 // 2) Illegal opcodes: must not be decoded
90 // -----
91 drive_if.drive_illegal_nop_test(500);
92
93 // Optional idle cycle
94 @(posedge clk);
95
96 // -----
97 // 3) Random test
98 // -----
99 repeat (100_000) begin
100     if (!tr.randomize()) $fatal("Randomize failed");
101     drive_if.drive_item(tr);
102 end
103
104 @(posedge clk);
105 $finish;
106 end
107
108 /* initial begin : shm_db
109 // Initial block to open shared memory and probe signals
110 $shm_open("shm_db");
111 $shm_probe("ASMTR");
112 end
113 */
114 endmodule

```

Draft

7.6 fv_microprocessor_top - module

```

1 // fv/fv_microprocessor_top.sv
2 `timescale 1ns/1ps
3 //`include "property_defines.svh"
4
5 //
6 // -----
7
8 // Module: fv_microprocessor_top
9 // Description:
10 //   Top-level SVA checker using microprocessor_if signals.
11 //   - Keeps the original assertion text and naming.
12 //   - Uses vif.* as the signal source.
13 // -----
14
15 module fv_microprocessor_top #(
16     parameter int DATA_W = 32,
17     parameter int DIR_W   = 5
18 )(
19     input  logic clk,
20     input  logic arst_n,
21     microprocessor_if.monitor vif
22 );

```

```

23 //
24 // -----
25 // ASSERTIONS
26 // -----
27 // =====
28 // PC AND PRF
29 'AST (it, pc_aligned,
30     1'b1 |->, ('PC_Q[1:0] == 2'b00)
31 )
32
33 'AST(it, next_instr,
34     ('OPCODE inside{ OPCODE_R_TYPE, OPCODE_I_TYPE, OPCODE_U_LUI,
35         OPCODE_U_AUIPC, OPCODE_L_TYPE, OPCODE_S_TYPE}) |->,
36     ('PC_NEXT == 'PC_Q + 3'b100)
37 )
38
39 'AST (it, x0_always_zero,
40     1'b1 |->, ('X0 == '0)
41 )
42 // =====
43                                     Draft
44 // TYPE_I INSTRUCTIONS
45 'AST (it, imm_I_type,
46     ('OPCODE == OPCODE_I_TYPE) |->, ('IMM_SEL == IMM_I)
47 )
48
49 'AST(it, prf_we,
50     ('OPCODE == OPCODE_I_TYPE) |->, ('RF_WE == 1'b1)
51 )
52
53 'AST(it, pc_4,
54     ('OPCODE == OPCODE_I_TYPE) |->, ('PC_NEXT == 'PC_Q + 3'b100)
55 )
56 // =====
57 // I-type: ADDI, SLTI, SLTIU, XORI, ORI, ANDI
58 // =====
59
60 'AST(it, addi_result,
61     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_ADD_SUB)) |->,
62     ('WB_DATA == ('ALU_OP1 + 'ALU_OP2))
63 )
64
65 /* 'AST(it, slti_result,
66     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_SLT)) |->,
67     ('WB_DATA ==
68     ( $signed('ALU_OP1) < $signed('ALU_OP2) ))
69 */

```

```

70
71
72 'AST(it, sltiu_result,
73     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_SLTU)) |->,
74     ('WB_DATA == (('ALU_OP1 < 'ALU_OP2) ? 32'd1 : 32'd0))
75 )
76
77 'AST(it, xori_result,
78     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_XOR)) |->,
79     ('WB_DATA == ('ALU_OP1 ^ 'ALU_OP2))
80 )
81
82 'AST(it, ori_result,
83     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_OR)) |->,
84     ('WB_DATA == ('ALU_OP1 | 'ALU_OP2))
85 )
86
87 'AST(it, andi_result,
88     (('OPCODE == OPCODE_I_TYPE) && ('FUNCT3 == F3_AND)) |->,
89     ('WB_DATA == ('ALU_OP1 & 'ALU_OP2))
90 )
91
92 // =====
93 // R-type: ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
94 // =====
95
96 'AST(it, sub,
97     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_ADD_SUB) && ('FUNCT7 == 7'
98         b0100_000)) |->, Draft
99     ('WB_DATA == ('RS1_DATA - 'RS2_DATA))
100 )
101
102 'AST(it, add,
103     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_ADD_SUB) && ('FUNCT7 == 7'
104         b0000_000)) |->,
105     ('WB_DATA == ('RS1_DATA + 'RS2_DATA))
106 )
107
108 'AST(it, sll,
109     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_SLL) && ('FUNCT7 == 7'
110         b0000_000)) |->,
111     ('WB_DATA == ('RS1_DATA << 'RS2_DATA[4:0]))
112 )
113
114 'AST(it, slt,
115     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_SLT) && ('FUNCT7 == 7'
116         b0000_000)) |->,
117     (('WB_DATA) == ($signed('RS1_DATA) < $signed('RS2_DATA) ? 32'd1 : 32'd0)
118 )
119
120 'AST(it, sltu,
121     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_SLTU) && ('FUNCT7 == 7'
122         b0000_000)) |->,
123     ('WB_DATA == ('RS1_DATA < 'RS2_DATA ? 32'd1 : 32'd0) )
124 )

```

```

120
121 'AST(it, xor_,
122     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_XOR) && ('FUNCT7 == 7'
123         b0000_000)) |->,
124     ('WB_DATA == ('RS1_DATA ^ 'RS2_DATA))
125 )
126
127 'AST(it, srl,
128     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_SRL_SRA) && ('FUNCT7 == 7'
129         b0000_000)) |->,
130     ('WB_DATA == ('RS1_DATA >> 'RS2_DATA[4:0]))
131 )
132
133 'AST(it, sra,
134     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_SRL_SRA) && ('FUNCT7 == 7'
135         b0100_000)) |->,
136     ($signed('WB_DATA) == ($signed('RS1_DATA) >>> 'RS2_DATA[4:0])) )
137 )
138
139 'AST(it, or_,
140     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_OR) && ('FUNCT7 == 7'
141         b0000_000)) |->,
142     ('WB_DATA == ('RS1_DATA | 'RS2_DATA))
143 )
144
145 'AST(it, and_,
146     (('OPCODE == OPCODE_R_TYPE) && ('FUNCT3 == F3_AND) && ('FUNCT7 == 7'
147         b0000_000)) |->,
148     ('WB_DATA == ('RS1_DATA & 'RS2_DATA'))
149 )
150
151 //
152 // =====
153
154 // JAL (JUMP AND LINK) INSTRUCTION
155 //
156 // =====
157
158 'AST(it, jal,
159     ('OPCODE == OPCODE_JAL_TYPE) |->,
160     ('PC_NEXT == ('PC_Q + 'ALU_OP2))
161 )
162
163 //
164 // =====
165
166 // LUI INSTRUCTION
167 //
168 // =====
169
170 'AST (it, lui,
171     ('OPCODE == OPCODE_U_LUI) |->,
172     ('WB_DATA == (32'd0 + 'IMM))
173 )
174
175

```

```

162 //
163 // AUIPC INSTRUCTION
164 //
=====
165 'AST (it, auipc,
166     ('OPCODE == OPCODE_U_AUIPC) |->,
167     ('WB_DATA == ('PC_Q + 'IMM))
168 )
169 //
170 //
=====
171 // TYPE B INSTRUCTIONS
172 //
=====
173
174 'AST(it, b_instr,
175     ('OPCODE == OPCODE_B_TYPE) |->,
176     (('BR_TAKEN) ? ('PC_NEXT == 'PC_Q + 'IMM): ('PC_NEXT == 'PC_Q + 3'b100))
177 )
178
179 'AST(it, beq,
180     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BEQ) ) |->,
181     (('RS1_DATA == 'RS2_DATA) ? ('PC_NEXT == 'PC_Q + 'IMM): ('PC_NEXT == '
182         PC_Q + 3'b100)) Draft
183 )
184
185 'AST(it, bne,
186     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BNE) ) |->,
187     (('RS1_DATA != 'RS2_DATA) ? ('PC_NEXT == 'PC_Q + 'IMM): ('PC_NEXT == '
188         PC_Q + 3'b100))
189 )
190
191 'AST(it, blt,
192     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BLT) ) |->,
193     (($signed('RS1_DATA) < $signed('RS2_DATA)) ? ('PC_NEXT == 'PC_Q + 'IMM)
194         : ('PC_NEXT == 'PC_Q + 3'b100))
195 )
196
197 'AST(it, bge,
198     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BGE) ) |->,
199     (($signed('RS1_DATA) >= $signed('RS2_DATA)) ? ('PC_NEXT == 'PC_Q + 'IMM)
200         : ('PC_NEXT == 'PC_Q + 3'b100))
201 )
202
203 'AST(it, bltu,
204     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BLTU) ) |->,
205     (('RS1_DATA < 'RS2_DATA) ? ('PC_NEXT == 'PC_Q + 'IMM): ('PC_NEXT == '
206         PC_Q + 3'b100))
207 )
208
209 'AST(it, bgeu,

```

```

205     (('OPCODE == OPCODE_B_TYPE) && ('FUNCT3 == F3_BGEU) ) |->,
206     (('RS1_DATA >= 'RS2_DATA) ? ('PC_NEXT == 'PC_Q + 'IMM): ('PC_NEXT == '
        PC_Q + 3'b100))
207 )
208
209 //
        =====
210 //  TYPE S INSTRUCTIONS
211 //
        =====
212 'AST(it,dmem_we,
213     ('OPCODE == OPCODE_S_TYPE) |->,
214     ('DMEM_WE == 1'b1)
215 )
216
217 'AST(it, s_type,
218     ('OPCODE == OPCODE_S_TYPE) |->,
219     (('DMEM_ADDR == 'RS1_DATA + 'IMM) && ('DMEM_WDATA == 'RS2_DATA))
220 )
221
222 //
        =====
223 //  TYPE L INSTRUCTIONS
224 //
        =====
                                Draft
225 'AST(it,write_back_from_dmem,
226     ('OPCODE == OPCODE_L_TYPE) |->,
227     (('DMEM_WE == 1'b0) && ('RF_WE == 1'b1))
228 )
229
230 'AST(it, data_to_prf,
231     ('OPCODE == OPCODE_L_TYPE) |->,
232     ('DMEM_ADDR == 'RS1_DATA + 'IMM )
233 )
234
235 //
        -----
236 // COVERS
237 //
        -----
238 'COV(it, cov_any_i_type, ('OPCODE == OPCODE_I_TYPE) |->, 1'b1)
239 'COV(it, cov_any_r_type, ('OPCODE == OPCODE_R_TYPE) |->, 1'b1)
240 'COV(it, cov_any_b_type, ('OPCODE == OPCODE_B_TYPE) |->, 1'b1)
241 'COV(it, cov_any_l_type, ('OPCODE == OPCODE_L_TYPE) |->, 1'b1)
242 'COV(it, cov_any_s_type, ('OPCODE == OPCODE_S_TYPE) |->, 1'b1)
243 'COV(it, cov_any_lui,     ('OPCODE == OPCODE_U_LUI) |->, 1'b1)
244 'COV(it, cov_any_auipc,   ('OPCODE == OPCODE_U_AUIPC)|->, 1'b1)
245 'COV(it, cov_any_jal,     ('OPCODE == OPCODE_JAL_TYPE)|->, 1'b1)
246

```

```

247 //
248 // COVERGROUPS
249 //
250
251
252 // Sample coverage on every cycle after reset
253 covergroup cg_instr @(posedge clk);
254     option.per_instance = 1;
255
256     // Cover that we ever saw a NOP on the instruction bus
257     cp_nop: coverpoint ('OPCODE == 7'b1111111) iff (arst_n) {
258         bins seen_nop = {1'b1};
259     }
260
261     // Basic opcode coverage (expand with more bins later)
262     cp_opcode: coverpoint 'OPCODE iff (arst_n) {
263         bins r_type = {OPCODE_R_TYPE};
264         bins i_type = {OPCODE_I_TYPE};
265         bins l_type = {OPCODE_L_TYPE};
266         bins s_type = {OPCODE_S_TYPE};
267         bins b_type = {OPCODE_B_TYPE};
268         bins lui    = {OPCODE_U_LUI};
269         bins auipc  = {OPCODE_U_AUIPC};
270         bins jal    = {OPCODE_JAL_TYPE};
271         // If your CU collapses illegal opcodes into NOP, you can still track "
           // Draft
           // others"
272         bins other = default;
273     }
274
275     // Funct3 only makes sense for some opcodes; use conditional bins
276     cp_funct3_i: coverpoint 'FUNCT3 iff (arst_n && ('OPCODE == OPCODE_I_TYPE))
277     {
278         bins addi = {F3_ADD_SUB};
279         bins slti = {F3_SLT};
280         bins sltiu = {F3_SLTU};
281         bins xori = {F3_XOR};
282         bins ori  = {F3_OR};
283         bins andi = {F3_AND};
284     }
285
286     cp_funct3_r: coverpoint 'FUNCT3 iff (arst_n && ('OPCODE == OPCODE_R_TYPE))
287     {
288         bins add_sub = {F3_ADD_SUB};
289         bins sll      = {F3_SLL};
290         bins slt      = {F3_SLT};
291         bins sltu     = {F3_SLTU};
292         bins xor_     = {F3_XOR};
293         bins srl_sra  = {F3_SRL_SRA};
294         bins or_      = {F3_OR};
295         bins and_     = {F3_AND};
296     }

```

```

296 // Funct7 only meaningful for R-type (SUB/SRA)
297 cp_funct7_r: coverpoint 'FUNCT7 iff (arst_n && ('OPCODE == OPCODE_R_TYPE))
    {
298     bins base = {7'b0000_000};
299     bins sub  = {7'b0100_000};
300 }
301
302 // R-type funct3 x funct7 (ensures SUB/SRA paths get hit)
303 cx_rtype: cross cp_funct3_r, cp_funct7_r iff (arst_n && ('OPCODE ==
    OPCODE_R_TYPE));
304
305 endgroup
306
307 cg_instr cg_instr_i;
308
309 initial begin
310     // Create covergroup instance
311     cg_instr_i = new();
312 end
313
314 endmodule

```

Draft